Algorithms and Data Structures 2.

–

Graph representations

Harrach Nóra - harrachnora@inf.elte.hu

Practice 1

Table of Contents

Basic Notation

Graph representations

Exercises

Table of Contents

Basic Notation

Graph representations

Exercises

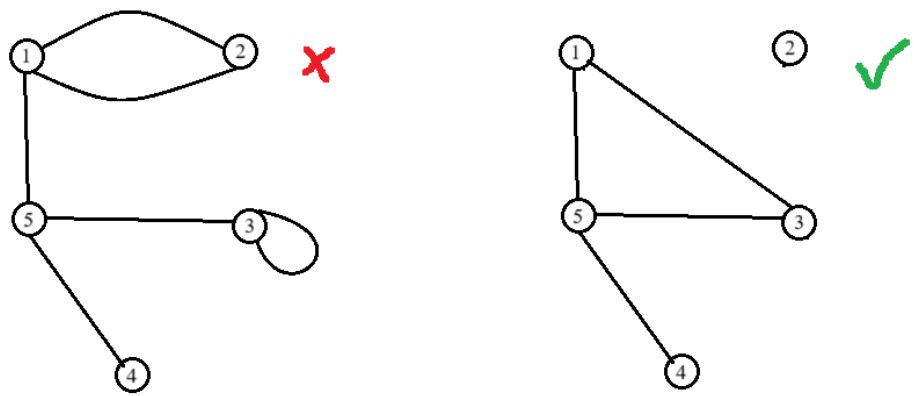
Basic Notation

Definition: A **graph** is a $G = (V, E)$ ordered pair, where V is the finite set of vertices, and $E \subseteq V \times V \setminus \{(u, u) : u \in V\}$ is the set of edges.

If $V = \{\}$ then the graph is **empty**.
The **vertices** of a graph can also be called **nodes**.

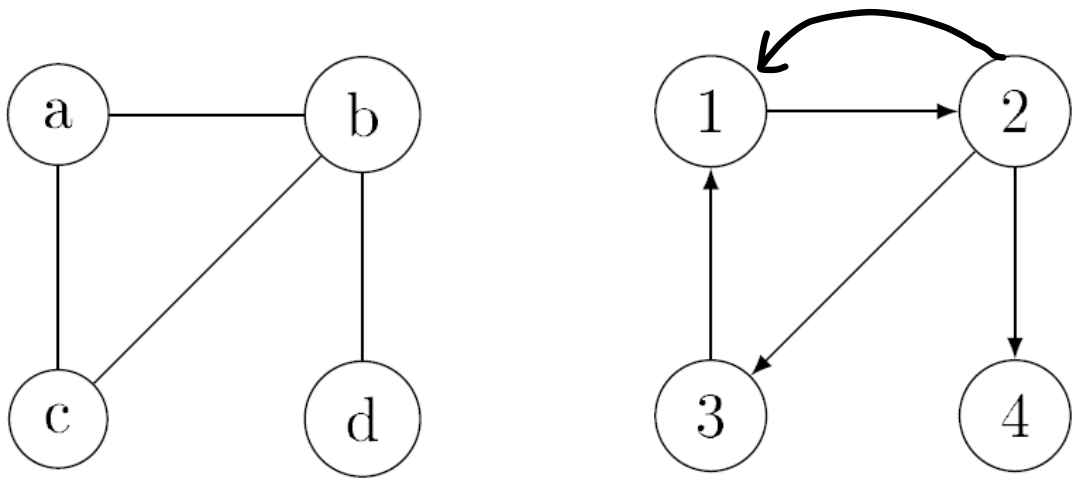
We will usually work with simple graphs, so

- no self-loops
- no parallel edges



Undirected graph: the edge $(u, v) \in E$ is the same as the edge (v, u).

Directed graph or **digraph**: edges (u, v) and (v, u) are not the same. In graphical representation: we will draw an arrow pointing from the first node to the second node



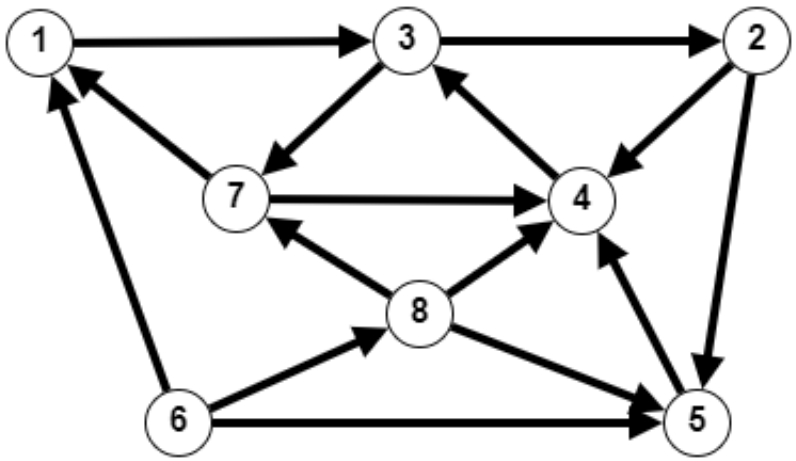
Path

Definition. In a given graph the sequence of vertices $\langle u_0, u_1, u_2, \ldots, u_n \rangle$ is a **path** if (u_{i-1}, u_i) is an edge of the graph for every $i \in 1..n$.

The **length** of this path is n (the number of edges).

Definition. The **subpath** of a path $\langle u_0, u_1, u_2, \ldots, u_n \rangle$ is a contiguous subsequence $\langle u_i, u_{i+1}, \ldots, u_j \rangle$ with $0 \leq i \leq j \leq n$.

Find paths and subpaths of these paths in the following graph:



<3,2,5,4>

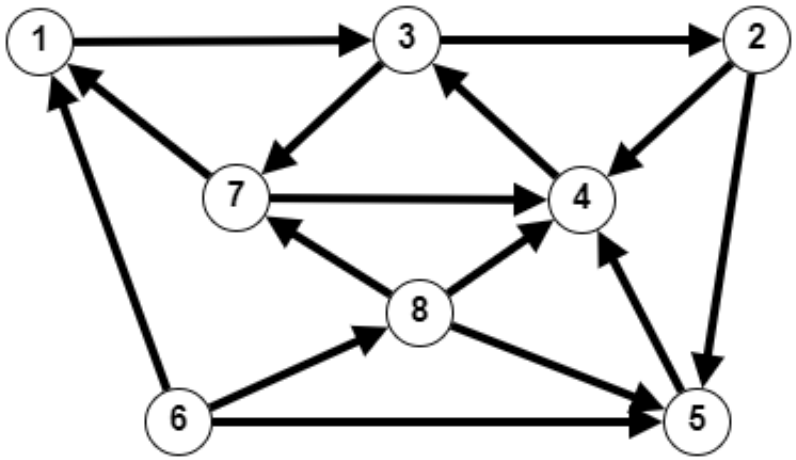
<6,1,3,2,5,4>

Loops

Definition. A path $\langle u_0, u_1, u_2, \ldots, u_n \rangle$ is a **loop** if $u_0 = u_n$ and the edges of the path are pairwise distinct.

Definition. A loop $\langle u_0, u_1, u_2, \ldots, u_{n-1}, u_0 \rangle$ is a **simple loop** if the vertices $u_0, u_1, \ldots u_{n-1}$ are pairwise distinct.

Find loops and simple loops in the following graph:



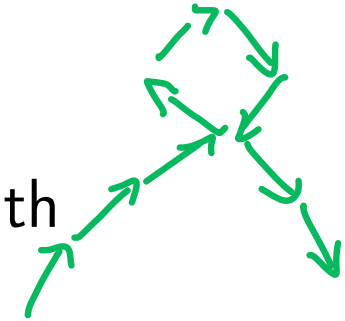
<3,2,4,3> loop

<3,2,4,3,2,4,3> not loop

simple loop

not simple loop: <1,3,2,4,3,7,1>

Loops and DAGs



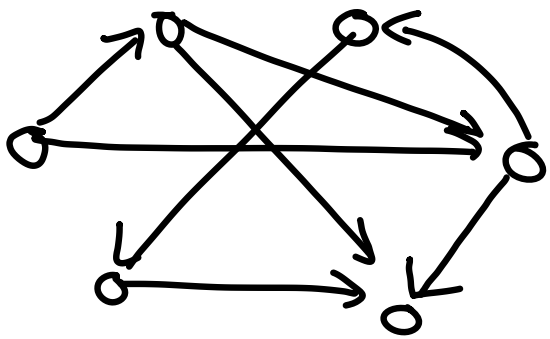
Definition. A path **contains a loop** if it has a subpath which is a loop.

Definition. A path is **acyclic** if it does not contain a loop.

Definition. A graph is **acyclic** if all the paths of the graph are acyclic (or in other words: it does not contain a path that is a loop).

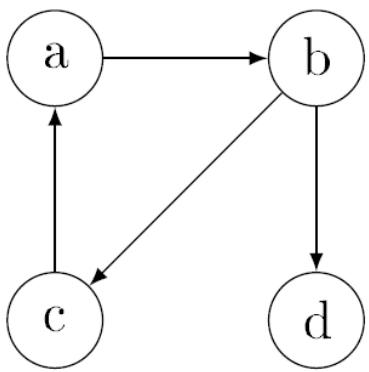
Definition. A DAG is a directed acyclic graph.

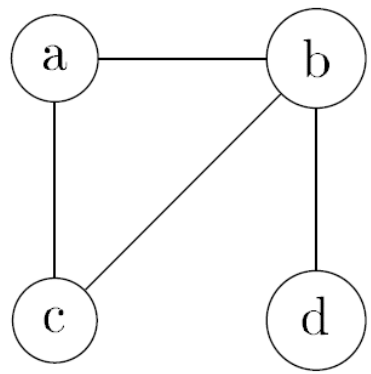
Draw examples of DAGs, both directed ~~and undirected~~.

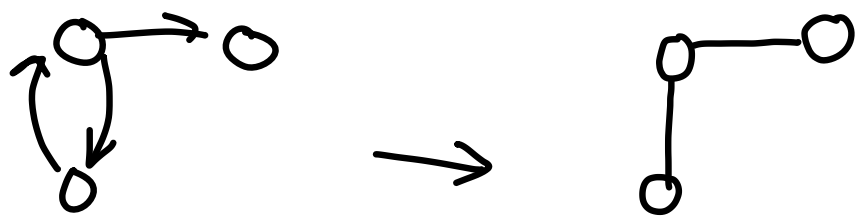


Undirected pair of a digraph

Definition. The **undirected pair** of a directed graph $G = (V, E)$ is the undirected graph $G' = (V, E')$ where $E' = \{(u, v) : (u, v) \in E \vee (v, u) \in E\}$.





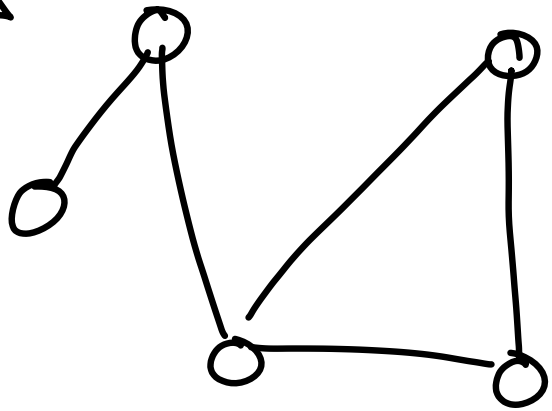


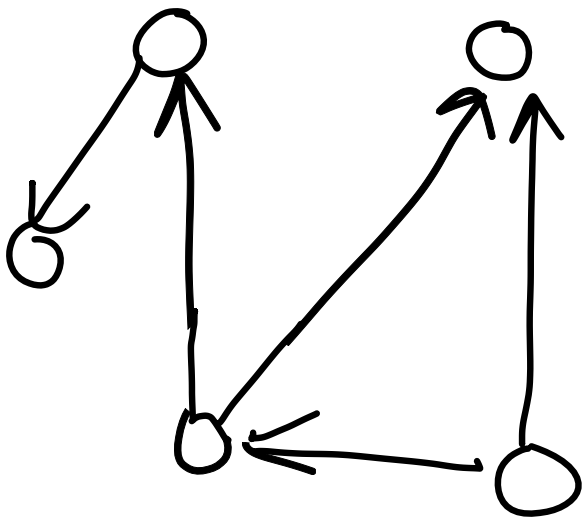
Connected graphs

Definition. An undirected graph is **connected**, if there is some path between each pair of vertices.

Definition. A directed graph is **connected**, if its undirected pair is connected.

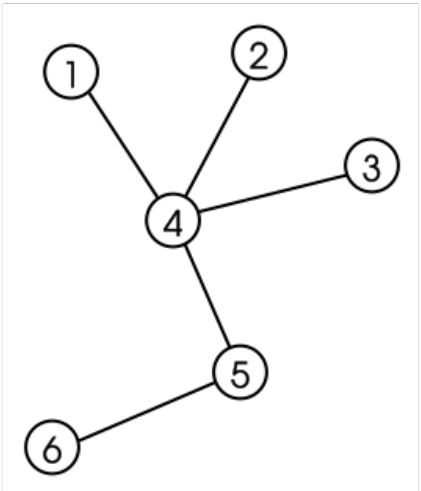
Examples:





Undirected trees

Definition. An undirected graph is a **tree** if it is connected and acyclic.



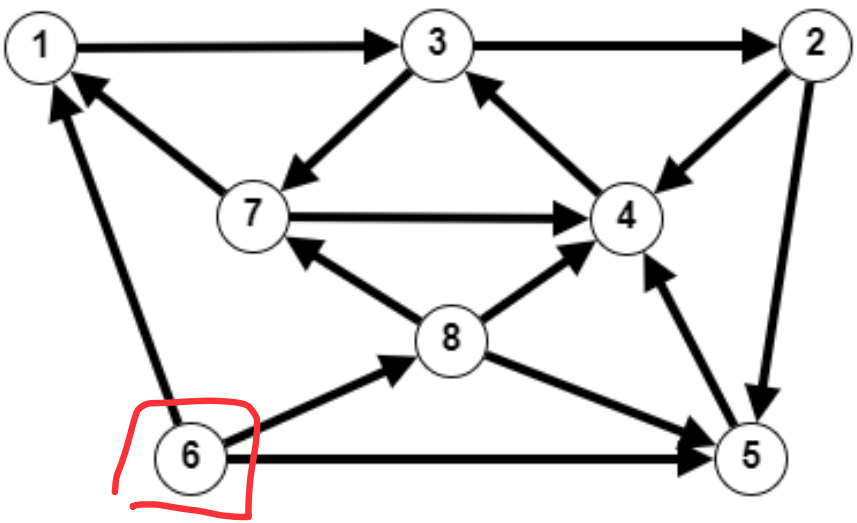
- A tree on n nodes has exactly $n-1$ edges.
- A graph is a tree $\longleftrightarrow$ it is a connected graph which will cease to be connected if we remove any edge from it.
- A graph is a tree $\longleftrightarrow$ it is an acyclic graph which will cease to be acyclic if we add an edge to it.

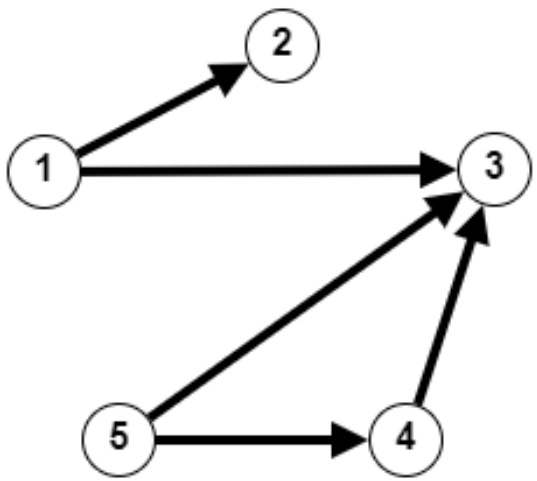
Directed trees

Definition. A vertex u of a directed graph is a **generator vertex** if every other vertex v is available from u, i.e. there is some path $u \rightsquigarrow v$.

Property.

- If a directed graph has a generator node then it is connected.
- There are connected digraphs that have no generator node.

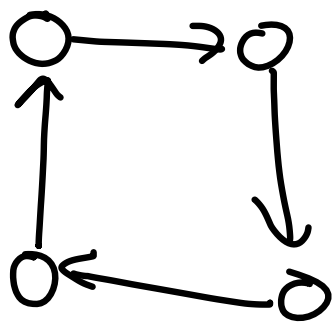




Directed trees

Property.

- There can be more than one generator vertices in a directed graph.
- If there are more than one generator vertices, then there is a loop in the graph, so it is not acyclic.

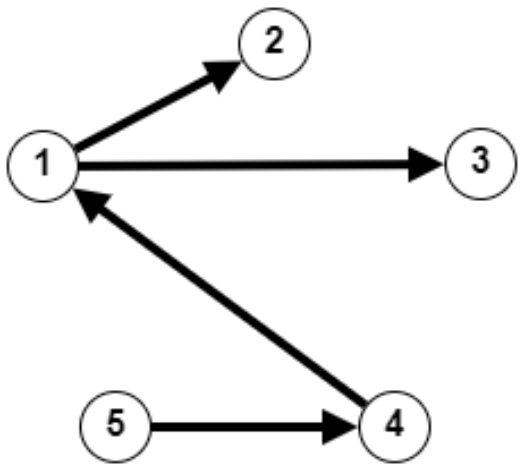


Directed trees

Definition. A directed graph is a **rooted tree** if it has a generator node, and its undirected pair is an acyclic graph. The generator vertex of a rooted tree is called its **root**.

Definition. A directed graph T is a **directed tree** if it is a rooted tree or empty.

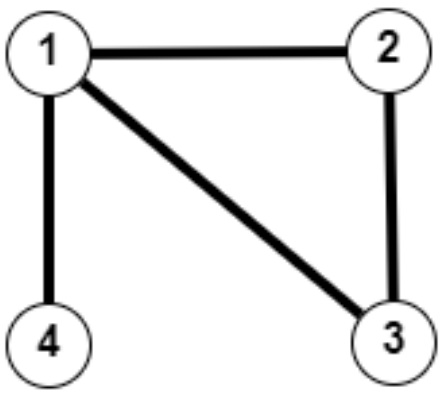
Property. A nonempty directed tree on n vertices has $n - 1$ edges.

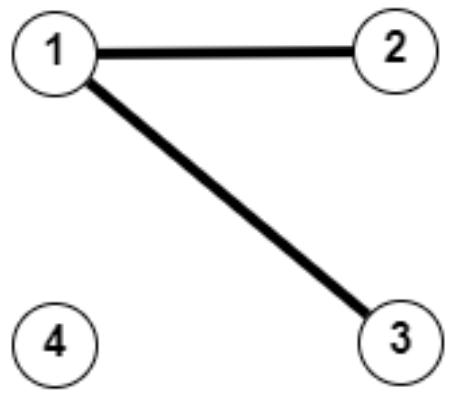


Subgraphs

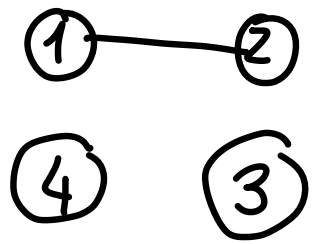
Definition. A graph $G' = (V', E')$ is a **subgraph** of the graph $G = (V, E)$ if $V' \subseteq V$ and $E' \subseteq E$ and both graphs are directed, or both graphs are undirected.

G' is a **proper subgraph** of G if $G' \neq G$ and G' is nonempty.

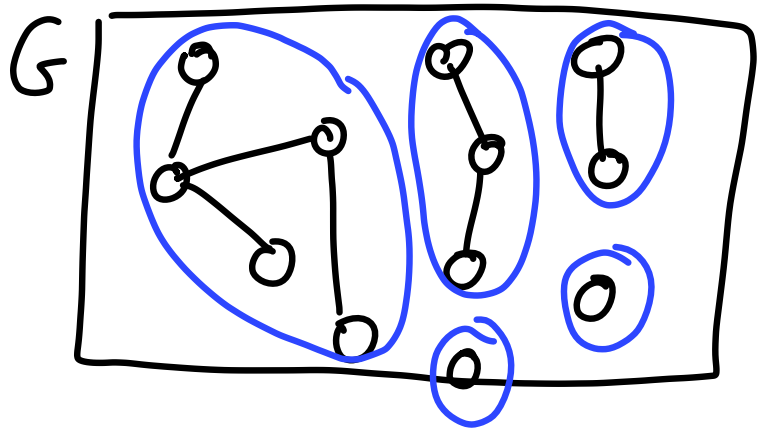




Definition. Two subgraphs are disjoint if they have no common vertex.



Connected components



Definition. A nonempty graph G' is a **connected component** of the graph G if

- G' is a connected subgraph of G and
- there is no connected subgraph G'' of G such that G' is a proper subgraph of G''.

Property. Every graph is either connected or consists of pairwise disjoint connected components.

Forests

Definition. A graph is a **forest** if its connected components are trees (or if it is a tree itself).

Property.

- And undirected graph is a forest $\iff$ it is acyclic,
- A directed graph G is a forest $\iff$ its undirected pair is acyclic and each connected component of G has a generator vertex.

Table of Contents

Basic Notation

Graph representations

Exercises

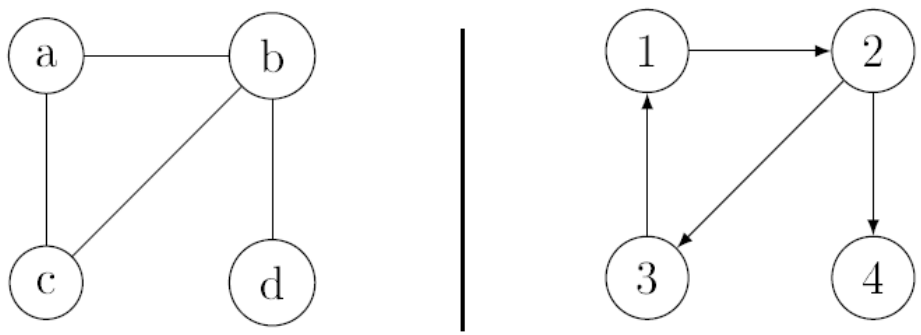
Graph representations

We will be using the following graph representations:

- graphical representation
- textual representation
- adjacency matrix representation
- adjacency list representation

Graphical representation

By **graphical representation** we mean a picture of the graph. Nodes are usually represented by circles, edges are represented by lines (undirected case) or arrows (directed case). Nodes are usually labelled with numbers or lowercase letters.

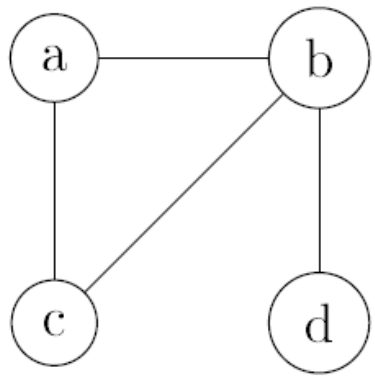


Textual representation

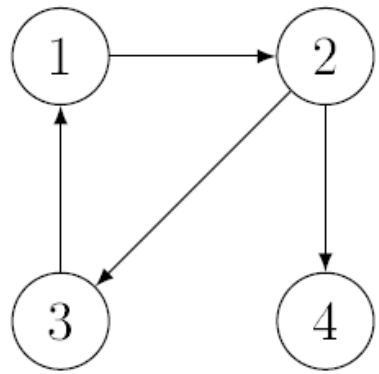
We list the nodes of the graph, and beside every node we list its **neighbours** (undirected graph) or **children** (directed graph).

In case of undirected graphs each edge would be represented twice (once at each end-node). Sometimes we only write one of those.

a – b ; c.
b – a ; c ; d.
c – a ; b.
d – b.



a – b ; c.
b – c ; d.



1 → 2.
2 → 3 ; 4.
3 → 1.

1 → 2.
2 → 3 ; 4.
3 → 1.
4 → .

Adjacency matrix representation

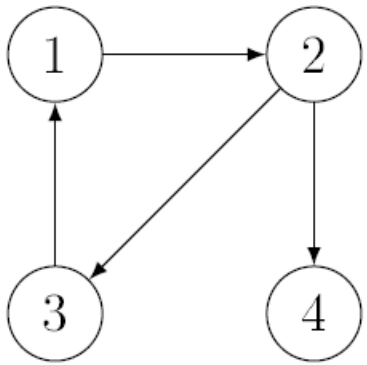
We represent graph $G = (V, E)$ with $V = \{v_1, v_2, \ldots, v_n\}$ with a bit matrix

$$A/1 : bit[n, n]$$

with the following rule:

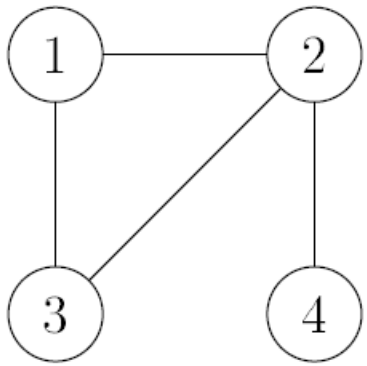
$$A[i, j] = 1 \iff (v_i, v_j) \in E$$
$$A[i, j] = 0 \iff (v_i, v_j) \notin E$$

Example:



A	1	2	3	4
1	0	1	0	0
2	0	0	1	1
3	1	0	0	0
4	0	0	0	0

Adjacency matrix representation



A	1	2	3	4
1	0	1	1	0
2	1	0	1	1
3	1	1	0	0
4	0	1	0	0

Space complexity:

- Space needed to store this matrix: n^2 bits
- in the main diagonal: all 0 bits
- for undirected graphs the matrix is symmetrical
- it would be enough to store "half of the matrix" (the triangle under the main diagonal)
- $\frac{n^2-n}{2}$ bits would be enough, but that is also $\Theta(n^2)$
- Space complexity: $\Theta(n^2)$

Adjacency list representation

We represent graph $G = (V, E)$ with $V = \{v_1, v_2, \ldots, v_n\}$ with a pointer array

$$A/1 : Edge^*[n]$$

such that for $i \in 1..n$ the pointer A[i] points at an S1L list which contains the neghbours (if G is undirected) or the children (if G is directed) of the vertex v_i.

$Edge$
$+v : \mathbb{N}$
$+next : Edge*$

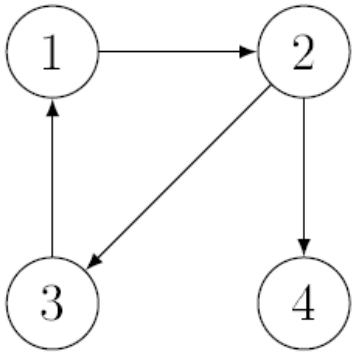
$p \to v$

$p \to next$

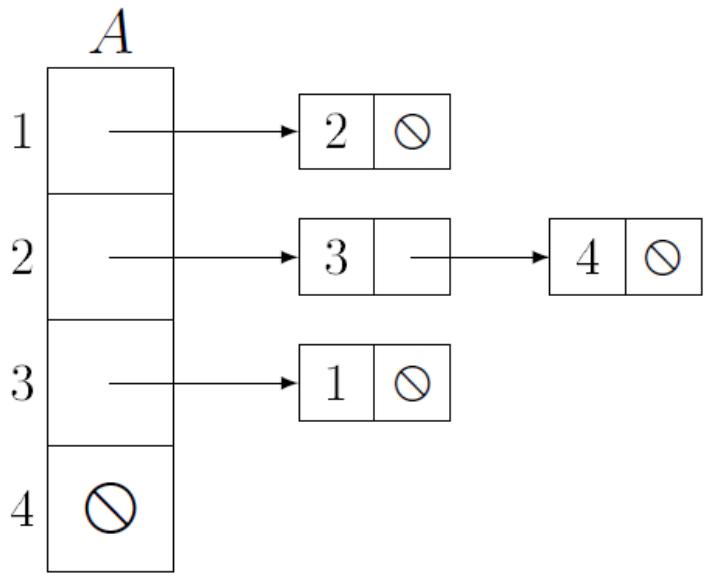
$\begin{pmatrix} p.v \\ p.next \end{pmatrix}$

We can use other types of lists (for example C2L).

Adjacency list representation



$A[1] \rightarrow v = 2$; $A[1] \rightarrow next = \oslash$
$A[2] \rightarrow v = 3$; $A[2] \rightarrow next \rightarrow v = 4$
; $A[2] \rightarrow next \rightarrow next = \oslash$
$A[3] \rightarrow v = 1$; $A[3] \rightarrow next = \oslash$
$A[4] = \oslash$



Adjacency list representation

Throughout this semester we will be using notation $|V| = n$ and $|E| = m$ for the number of vertices and edges of a graph.

Space complexity:

- In undirected graphs all edges are represented twice in the adjacency list representation.
- We have a pointer array $A/1 : Edge^*[n]$ to store and the S1L lists which have alltogether m (directed case) or $2m$ (undirected case) elements.
- Space compelxity: $\Theta(n + m)$

Sparse of dense graphs

We say that a graph is **sparse** if it has relatively few edges compared to the number n of vertices, so $m \in O(n)$.

We say that a graph is **dense** if it has many edges, $m \in \Theta(n^2)$.

- In case of sparse graphs the space complexity of adj. list representation is $\Theta(m + n) = \Theta(n)$ which is significantly better than the space complexity of the adj. mtx representetation $\Theta(n^2)$.
- For dense graphs the space complexity of both representations is $\Theta(n^2)$.

Which representation is better?

If we want to decide if $(v_i, v_j) \in E$ holds or not (is it an edge?), then

- in an adjacency list representation we have to search the S1L list $A[i]$ for index j, which can take $deg(i)$ steps
- in an adjacency matrix representation we have to check if $A[i, j]$ is 0 or 1, which is only one step.

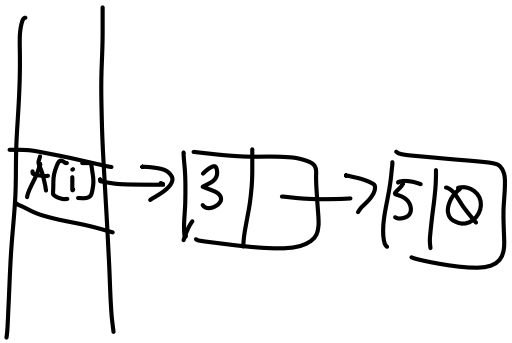
If we perform many such operations in an algorithm, then adj. matrix representation is a better choice.

Which representation is better?

If we want to list all the neighbours (undirected case) or children (directed case) of a vertex v_i then

- in an adjacency list representation we just have to print all elements of the S1L list $A[i]$ which is only $deg(i)$ steps
- in an adjacency matrix representation we have to check all elements $A[i, j]$ for j=1..n, which takes n steps.

If we perform many such operations in an algorithm, then adj. list representation is a better choice.



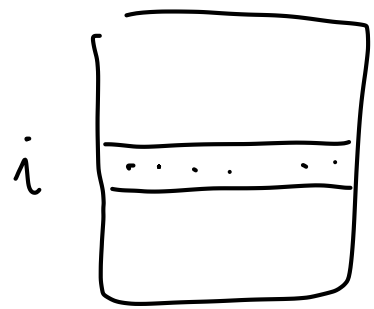


Table of Contents

Basic Notation

Graph representations

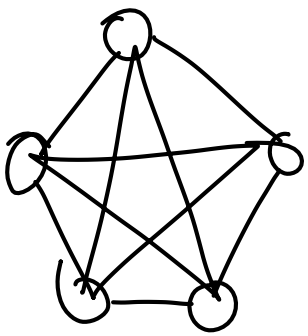
Exercises

Exercises

1. Design an algorithm which produces the (a) adjacency matrix, (b) adjacency list representation of the complete graph K_n for a given n.
2. Design an algorithm which produces the adjacency list representation of a graph that is given in adjacency matrix representation.
3. Design an algorithm which produces the adjacency matrix representation of a graph that is given in adjacency list representation.
4. Design an algorithm which produces the adjacency list representation of the complement of the graph G which is given with adjacency list representation. We may suppose that all the S1L lists are sorted according to the v values.

Exercises

1. Design an algorithm which produces the (a) adjacency matrix, (b) adjacency list representation of the complete graph K_n for given n.



0	1	1	1	1
1	0	1	1	1
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

```
adjMtx(n:ℕ): A:ℕ[n,n]
A:ℕ[n,n]
i=1 to n
    j=1 to n
        i=j
        A[i,j]=0 | A[i,j]=1
return A
```

Exercises

2. Design an algorithm which produces the adjacency list representation of a graph that is given in adjacency matrix representation.

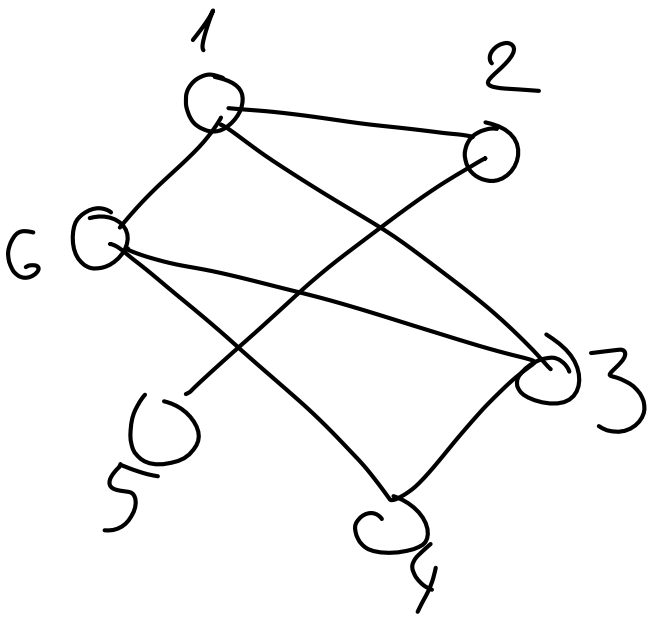
Exercises

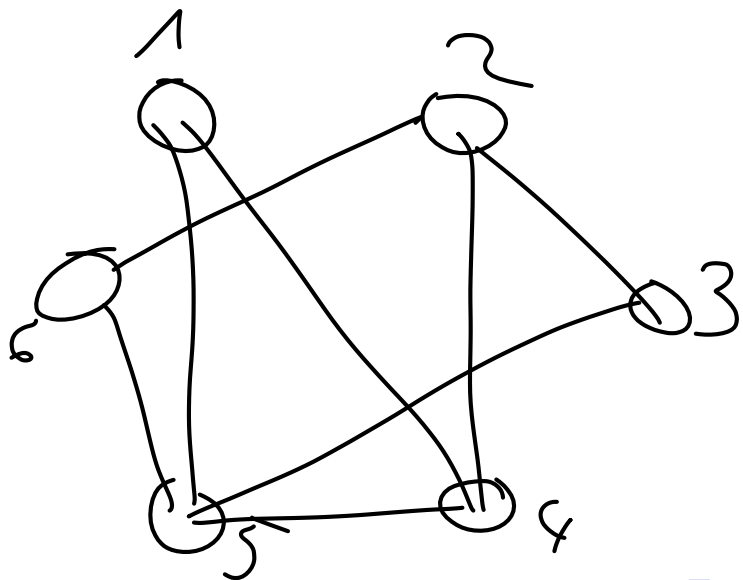
3. Design an algorithm which produces the adjacency matrix representation of a graph that is given in adjacency list representation.

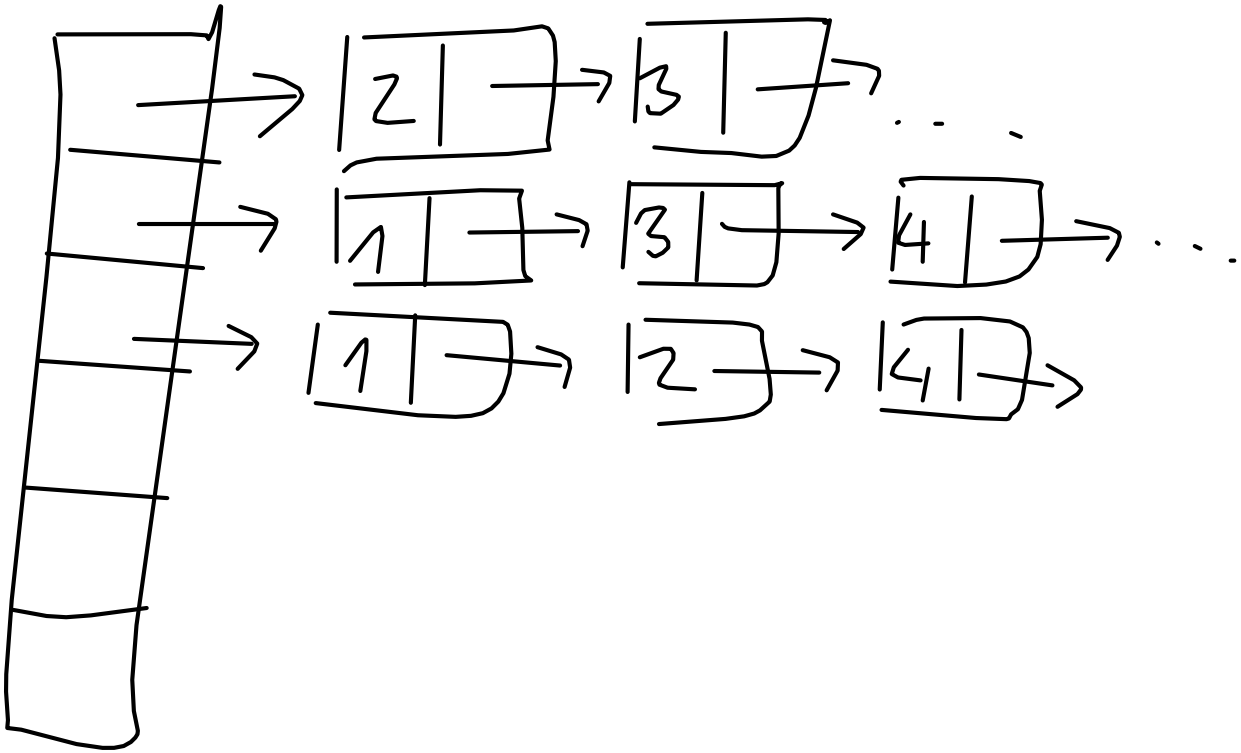
Exercises

4. Design an algorithm which produces the adjacency list representation of the complement of the graph G which is given with adjacency list representation. We may suppose that all the S1L lists are sorted according to the v values. (The compement of the graph $G = (V, E)$ is the graph $G' = (V, E')$ with

$$E' = \{(u, v) : (u, v) \notin E\}$$






2
3
1
3
4
1
2
4